

Assignment 4 - Dictionary Assignment

Every question is shown directly above its answer.

Q1. What is a dictionary in Python?

Answer:

A mutable collection of unique keys mapped to values.

Q2. Create a dictionary for Name: John, Age: 21, Department: CSE.

Answer:

```
student = {"Name": "John", "Age": 21, "Department": "CSE"}
```

Q3. How do you access the value of a key named "Age"?

Answer:

`student["Age"]` returns 21.

Q4. What happens when you access a key that does not exist, such as `student["Marks"]`?

Answer:

Square-bracket access raises `KeyError`. `student.get("Marks")` returns `None` unless a default is supplied.

Q5. What is the difference between `student["Age"]` and `student.get("Age")`?

Answer:

Both return the value for an existing key. Missing keys raise `KeyError` with brackets but return `None` (or a supplied default) with `get()`.

Q6. Write a program to print all keys of a dictionary.

Answer:

```
for key in student:
    print(key)
```

Q7. Write a program to print all values.

Answer:

```
for value in student.values():
    print(value)
```

Q8. Write a program to print every key-value pair (Name : John, Age : 21, Department : CSE).

Answer:

```
for key, value in student.items():
    print(f"{key} : {value}")
```

Q9. How do you check whether a key exists in a dictionary?

Answer:

Use `"Age" in student`.

Q10. How do you find the number of items in a dictionary?

Answer:

Use `len(student)`.

Q11. Add City : Chennai to an existing dictionary.

Answer:

```
student["City"] = "Chennai"
```

Q12. Update the age from 21 to 22.

Answer:

```
student["Age"] = 22
```

Q13. Delete the key "Department".

Answer:

```
del student["Department"] (or student.pop("Department", None) if it may be missing).
```

Q14. Remove the last inserted key-value pair.

Answer:

```
student.popitem() removes and returns the last-inserted pair in modern Python.
```

Q15. Clear the entire dictionary.

Answer:

```
student.clear()
```

Q16. Create a dictionary from keys=["A","B","C"] and values=[10,20,30].

Answer:

```
keys = ["A", "B", "C"]
values = [10, 20, 30]
result = dict(zip(keys, values))
# {"A": 10, "B": 20, "C": 30}
```

Q17. Merge two dictionaries.

Answer:

```
merged = first.copy()
merged.update(second)
# Values in second replace values for duplicate keys.
```

Q18. Copy one dictionary into another.

Answer:

```
copy_of_student = student.copy() # shallow copy
```

Q19. Find the maximum value in marks={"A":85,"B":92,"C":76}.

Answer:

```
maximum = max(marks.values()) # 92
```

Q20. Find the student with the highest marks (expected: B scored 92).

Answer:

```
name, score = max(marks.items(), key=lambda item: item[1])
print(f"{name} scored {score}")
```

Q21. Print only the keys.

Answer:

```
print(marks.keys())
```

Q22. Print only the values.

Answer:

```
print(marks.values())
```

Q23. Print all key-value pairs using items().

Answer:

```
for name, score in marks.items():  
    print(name, score)
```

Q24. Count how many values are greater than 50.

Answer:

```
count = sum(value > 50 for value in marks.values())
```

Q25. Print only students who scored above 80.

Answer:

```
for name, score in marks.items():  
    if score > 80:  
        print(name, score)
```

Q26. Calculate the average of all values.

Answer:

```
average = sum(marks.values()) / len(marks) # when non-empty
```

Q27. Count how many keys begin with the letter "A".

Answer:

```
count = sum(key.startswith("A") for key in marks)
```

Q28. Explain the use of keys().

Answer:

Returns a dynamic view of a dictionary's keys.

Q29. Explain the use of values().

Answer:

Returns a dynamic view of a dictionary's values.

Q30. Explain the use of items().

Answer:

Returns a dynamic view of (key, value) pairs, useful for iteration.

Q31. Explain the use of pop().

Answer:

pop(key) removes and returns a value; a default can be provided to avoid KeyError.

Q32. Explain the use of popitem().

Answer:

Removes and returns the last-inserted key-value pair in modern Python.

Q33. Explain the use of update().

Answer:

Adds pairs from another mapping; duplicate keys are overwritten.

Q34. Explain the use of setdefault().

Answer:

Returns the existing value, or inserts and returns the default if the key is absent.

Q35. Explain the use of fromkeys().

Answer:

Creates a dictionary from keys, assigning each the same value (None by default). Avoid a mutable shared default unless intended.

Q36. Count each character in "banana".

Answer:

```
text = "banana"
frequency = {}
for char in text:
    frequency[char] = frequency.get(char, 0) + 1
# {"b": 1, "a": 3, "n": 2}
```

Q37. Count words in "I love Python and I love AI".

Answer:

```
sentence = "I love Python and I love AI"
frequency = {}
for word in sentence.split():
    frequency[word] = frequency.get(word, 0) + 1
# I: 2, love: 2, Python: 1, and: 1, AI: 1
```

Q38. Reverse {"A":1,"B":2}.

Answer:

```
original = {"A": 1, "B": 2}
reversed_dict = {value: key for key, value in original.items()}
# {1: "A", 2: "B"}; repeated values overwrite earlier keys.
```

Q39. Find duplicate values in a dictionary.

Answer:

```
data = {"A": 1, "B": 2, "C": 1}
values = list(data.values())
duplicates = {value for value in values if values.count(value) > 1}
# {1}
```

Q40. Sort a dictionary based on keys.

Answer:

```
sorted_by_key = dict(sorted(data.items()))
```

Q41. Sort a dictionary based on values.

Answer:

```
sorted_by_value = dict(sorted(data.items(), key=lambda item: item[1]))
```

Q42. Create a dictionary comprehension for {1:1, 2:4, 3:9, 4:16, 5:25}.

Answer:

```
squares = {number: number ** 2 for number in range(1, 6)}
```

Q43. Create a dictionary of cubes from 1 to 10 using comprehension.

Answer:

```
cubes = {number: number ** 3 for number in range(1, 11)}
```

Q44. Convert two lists into a dictionary using dictionary comprehension.

Answer:

```
keys = ["A", "B", "C"]
values = [10, 20, 30]
result = {key: value for key, value in zip(keys, values)}
```

Q45. Create a dictionary for 1-20 labeling each value Even or Odd.

Answer:

```
labels = {n: ("Even" if n % 2 == 0 else "Odd") for n in range(1, 21)}
```

Q46. Create a dictionary for three student records based on the nested example.

Answer:

```
students = {101: {"Name": "John", "Age": 20}, 102: {"Name": "Alice", "Age": 21}, 103: {"Name": "Maria", "Age": 22}}
```

Q47. Print details of student 102.

Answer:

```
print(students[102])
```

Q48. Update the age of student 101.

Answer:

```
students[101]["Age"] = 21
```

Q49. Add marks to every student.

Answer:

```
marks_by_id = {101: 88, 102: 91, 103: 84}
for student_id, details in students.items():
    details["Marks"] = marks_by_id[student_id]
```

Q50. Print all student details using nested loops.

Answer:

```
for student_id, details in students.items():
    print(f"Student {student_id}:")
    for key, value in details.items():
        print(f"    {key}: {value}")
```

Q51. Given Jan=2500, Feb=3000, Mar=2700, Apr=4000, find the month with highest sales.

Answer:

```
sales = {"Jan": 2500, "Feb": 3000, "Mar": 2700, "Apr": 4000}
month, amount = max(sales.items(), key=lambda item: item[1])
# Apr, 4000
```

Q52. Create a dictionary mapping each word in "Python is amazing" to its length.

Answer:

```
sentence = "Python is amazing"
word_lengths = {word: len(word) for word in sentence.split()}
# {"Python": 6, "is": 2, "amazing": 7}
```

Q53. Given Pen=25, Pencil=50, Eraser=15, reduce each inventory quantity by 5.

Answer:

```
inventory = {"Pen": 25, "Pencil": 50, "Eraser": 15}
reduced = {item: quantity - 5 for item, quantity in inventory.items()}
# Pen: 20, Pencil: 45, Eraser: 10
```

Q54. Merge dictionaries and add values for duplicate keys: d1={"A":10,"B":20}, d2={"B":15,"C":5}.

Answer:

```
d1 = {"A": 10, "B": 20}
d2 = {"B": 15, "C": 5}
combined = d1.copy()
for key, value in d2.items():
    combined[key] = combined.get(key, 0) + value
# {"A": 10, "B": 35, "C": 5}
```

Q55. Create a phonebook that can add, search, update, delete, display contacts, and exit.

Answer:

```
def run_phonebook():
    phonebook = {}
    while True:
        print("\n1 Add  2 Search  3 Update  4 Delete  5 Display  6 Exit")
        choice = input("Choose an option: ").strip()
        if choice == "1":
            name = input("Name: ").strip()
            phonebook[name] = input("Phone number: ").strip()
        elif choice == "2":
            name = input("Name to search: ").strip()
            print(phonebook.get(name, "Contact not found"))
        elif choice == "3":
            name = input("Name to update: ").strip()
            if name in phonebook:
                phonebook[name] = input("New phone number: ").strip()
            else:
                print("Contact not found")
        elif choice == "4":
            name = input("Name to delete: ").strip()
            if name in phonebook:
                del phonebook[name]
            else:
                print("Contact not found")
        elif choice == "5":
            for name, number in phonebook.items():
                print(f"{name}: {number}")
            if not phonebook:
                print("Phonebook is empty")
        elif choice == "6":
            break
        else:
            print("Invalid option")

# Run with run_phonebook()
```